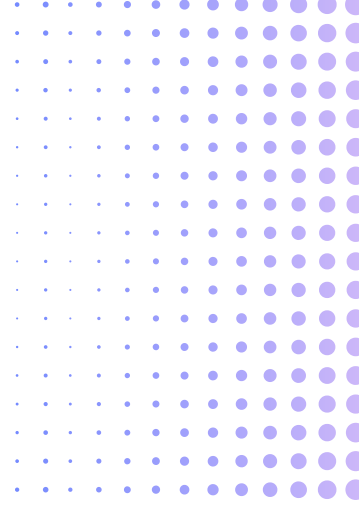




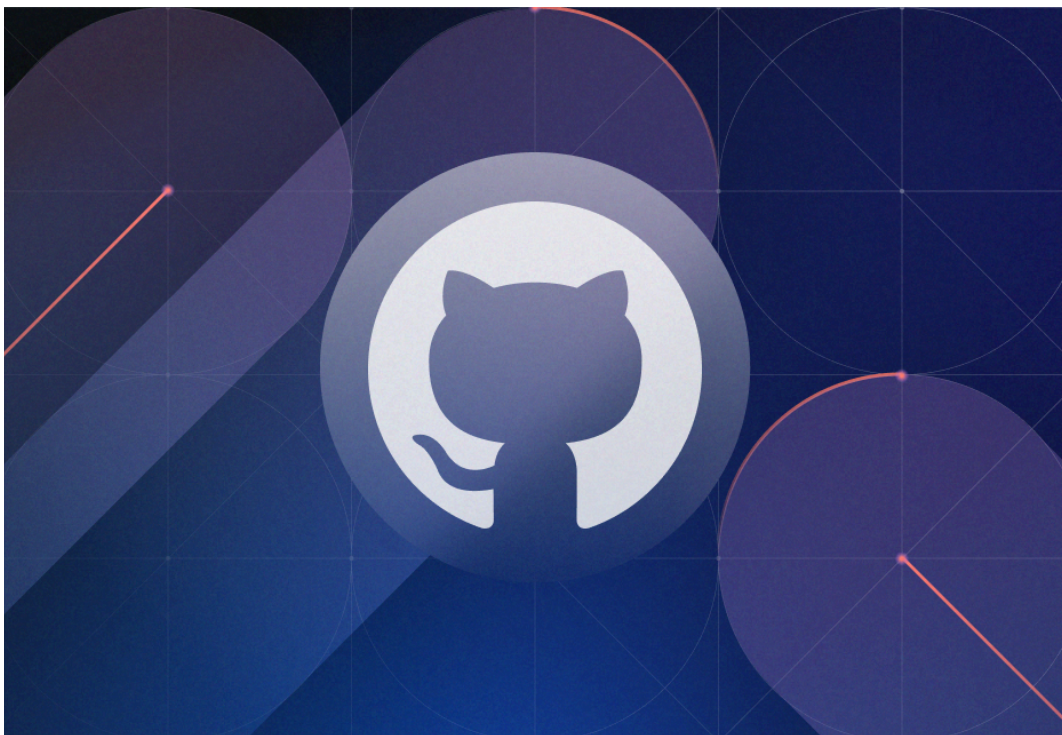
Manual para el manejo de Git y GitHub

Elaborado por:

Yáñez Soto Danna Lizeth
Universidad Autónoma de Quéretaro
Licenciatura en Actuaría

Introducción a GITHUB

Git, creado por Linus Torvalds (el fundador de Linux) en 2005, es un sistema de control de versiones distribuidas y de código abierto. Básicamente, se trata de un software diseñado para realizar un seguimiento de los cambios realizados en cualquier grupo de archivos. Como se distribuye, cada base de código, junto con su historial, está disponible para todos los usuarios de GitHub.



¿Qué es Git?

Git es un sistema de control de versiones distribuido (SCVD) que ayuda a los desarrolladores a gestionar su código. Realiza un seguimiento de los cambios y permite crear distintas versiones, o ramas, del código. Esto facilita el trabajo conjunto de los desarrolladores. Git también admite funciones como las áreas de preparación y los historiales de confirmaciones, que proporcionan un registro detallado de las modificaciones del código.

¿Qué es GitHub?

GitHub, por otro lado, proporciona funciones adicionales como control de acceso, seguimiento de errores, gestión de tareas y wikis, lo que facilita a los desarrolladores la colaboración en los proyectos. Al utilizar GitHub, puedes gestionar tu código, realizar un seguimiento de los cambios, revisar las contribuciones y debatir los problemas, todo en un mismo lugar. También se integra con diversas herramientas y servicios, lo que mejora el flujo de trabajo de desarrollo.

¿Cómo funcionan juntos?

Al cargar archivos en GitHub, los almacenas en un "repositorio de Git". Esto significa que al realizar cambios (o "compromisos") en los archivos de GitHub, Git se iniciará automáticamente para realizar un seguimiento y administrar los cambios.

Hay muchas acciones relacionadas con Git que puedes completar en GitHub directamente en el navegador, como crear un repositorio de Git, crear ramas y cargar y editar archivos.

Sin embargo, la mayoría de las personas trabajan en sus archivos localmente (en su propio ordenador), luego sincronizan continuamente estos cambios locales y todos los datos de Git relacionados, con el repositorio central "remoto" en GitHub.

Git frente a GitHub

Categoría	Git	GitHub
Definición	Sistema de control de versiones distribuido	Plataforma basada en web construida sobre Git
Propósito	Te ayuda a gestionar el código, seguir los cambios y crear ramas	Aloja repositorios Git y proporciona herramientas de colaboración adicionales
Características	Zonas de preparación, historiales de confirmaciones, bifurcaciones y fusiones	Control de acceso, seguimiento de errores, gestión de tareas, wikis e integraciones
Beneficio	Permite el trabajo colaborativo y el seguimiento detallado de los cambios en el código	Mejora los procesos de colaboración, gestión de proyectos y revisión de código

Git y GitHub son dos entidades diferentes que te ayudan a administrar y alojar archivos. En otras palabras Git sirve para controlar las versiones de los archivos mientras que GitHub es una plataforma para alojar tus repositorios Git.

¿Qué es el control de versiones?

El control de versiones es la práctica establecida de realizar cambios en el código de un proyecto y realizar un seguimiento de ellos. A medida que se agregan nuevas funciones al proyecto, el código continúa creciendo. Sin control de versiones, es difícil para el equipo de software saber cómo se revisó el código y qué cambios se realizaron.

El control de versiones también protege el código fuente de errores humanos y, como todo está en la nube, las posibilidades de que un desarrollador pierda su código fuente de GitHub son prácticamente nulas. Los sistemas de control de versiones como GitHub ayudan a garantizar que se realice un seguimiento de todas las revisiones del código.

Clasificación

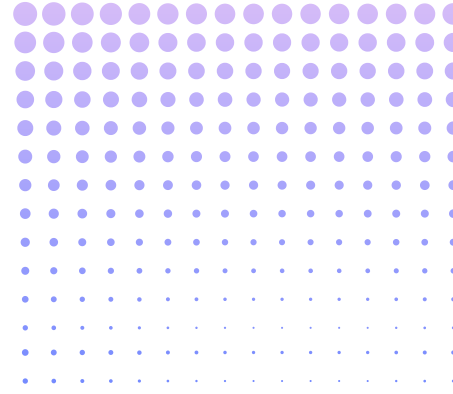
Podemos clasificar los sistemas de control de versiones atendiendo a la arquitectura utilizada para el almacenamiento del código:

Locales: Los cambios son guardados localmente y no se comparten con nadie. Esta arquitectura es la antecesora de las dos siguientes.

Centralizados: Existe un repositorio centralizado de todo el código, del cual es responsable un único usuario (o conjunto de ellos). Se facilitan las tareas administrativas a cambio de reducir flexibilidad, pues todas las decisiones fuertes (como crear una nueva rama) necesitan la aprobación del responsable.

Distribuidos : Cada usuario tiene su propio repositorio. Los distintos repositorios pueden intercambiar y mezclar revisiones entre ellos. Es frecuente el uso de un repositorio, que está normalmente disponible, que sirve de punto de sincronización de los distintos repositorios locales.

Conceptos clave



Branch (“rama”)

Una rama es una versión paralela de un repositorio. Se incluye en el repositorio, pero no afecta a la rama primaria o principal, lo que te permite trabajar libremente sin interrumpir la versión "activa".

Etiqueta ("tag")

Los tags permiten identificar de forma fácil revisiones importantes en el proyecto. Por ejemplo se suelen usar tags para identificar el contenido de las versiones publicadas del proyecto.

Cabeza ("head")

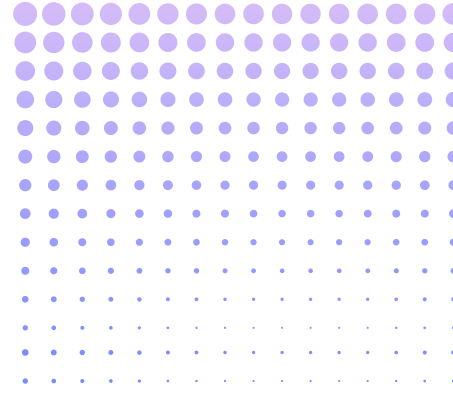
También a veces se llama tip (punta) y se refiere a la última confirmación, ya sea en el tronco ('trunk') o en una rama ('branch'). El tronco y cada rama tienen su propia cabeza, aunque HEAD se utiliza a veces libremente para referirse al tronco.

Desplegar ("checkout")

Es crear una copia de trabajo local desde el repositorio. Un usuario puede especificar una revisión en concreto u obtener la última. El término 'checkout' también se puede utilizar como un sustantivo para describir la copia de trabajo.



Conceptos clave



Confirmar ("commit")

Confirmar es escribir o mezclar los cambios realizados en la copia de trabajo del repositorio. Los términos 'commit' y 'checkin' también se pueden utilizar como sustantivos para describir la nueva revisión que se crea como resultado de confirmar.

Conflicto ("conflict")

Un conflicto se produce cuando diferentes partes realizan cambios en el mismo documento, y el sistema es incapaz de conciliar los cambios. Un usuario debe resolver el conflicto mediante la integración de los cambios, o mediante la selección de un cambio en favor del otro.

Tronco ("trunk")

La única línea de desarrollo que no es una rama (a veces también llamada línea base, línea principal o máster).

Fusionar, integrar, mezclar ("merge")

Una fusión o integración es una operación en la que se aplican dos tipos de cambios en un archivo o conjunto de archivos.



¿Cómo empezar con GITHUB?

Instalar Git

GitHub ofrece a los clientes de computadoras de escritorio que incluye una interfaz gráfica de usuario para las acciones de repositorio más comunes y una edición de línea de comando de actualización automática de Git para escenarios avanzados.

[Git Windows](#)

[GitHub Windows](#)

[Git Mac](#)

[GitHub Mac](#)

[Git para cualquier plataforma](#)

[Git para Linux](#)

Creación de una cuenta

Para empezar a trabajar con GitHub, deberá crear una cuenta personal gratuita y comprobar la dirección de correo electrónico.

- 1 [Ir a página de GitHub](#)
- 2 Hacer clic en registrarse
- 3 Seguir las indicaciones para crear la cuenta de usuario de Github
- 4 Verifica tu dirección de correo electrónico.

Crear un repositorio

Desde la pagina:

- 1 Accede a tu cuenta
- 2 Desde la página principal, haz clic en tu avatar en la esquina superior derecha y selecciona "Your repositories"
- 3 Haz clic en el botón verde que dice "New"
- 4 Configura tu nuevo repositorio:
 - Nombre
 - Descripción
 - Visibilidad
 - README*
 - Un README es un archivo que describe el propósito del proyecto.
- 5 Haga clic en Create repository

Desde la terminal

- 1 Abre la terminal de tu computadora
- 2 Usa el comando `cd` para cambiar al directorio de tu proyecto.

```
cd /Users/tu-usuario/ruta-al-proyecto
```

- 3 Ejecuta el comando para inicializar Git

```
git init
```



Comandos básicos

Crear y Clonar

Crear un nuevo repositorio **git init**

Clonar un repositorio local **git clone /path/to/repository**

Clonar un repositorio remoto **git clone username@host:/path/to/repository**

Agregar y Eliminar

Agregar cambios al índice **git add <filename>**

Agregar todos los cambios al índice **git add ***

Eliminar/borrar archivos **git rm <filename>**

Confirmar y Sincronizar

Confirmar cambios **git commit -m "Commit message"**

Enviar cambios al repositorio remoto **git push origin master**

Conectar un repositorio local a un repositorio remoto **git remote add origin <server>**

Actualizar el repositorio local con cambios remotos **git pull**

Ramas

Crear una nueva rama **git checkout -b <branch>**

Cambiar a la rama principal (master) **git checkout master**

Eliminar una rama **git branch -d <branch>**

Enviar una rama al repositorio remoto **git push origin <branch>**

Fusionar

Fusionar cambios de otra rama **git merge <branch>**

Ver cambios entre dos ramas **git diff <source_branch> <target_branch>**

Etiquetado

Crear una etiqueta **git tag <tag> <commit ID>**

Obtener IDs de confirmación **git log**

Restaurar

Reemplazar la copia de trabajo con la última versión desde HEAD **git checkout -- <filename>**

Guía rápida de comandos

CONFIGURAR HERRAMIENTAS

Configura la información del usuario para todos los repositorios locales

Establece el nombre que desea esté anexado a sus transacciones de commit

```
$ git config --global user.name "[name]"
```

Establece el e-mail que desea esté anexado a sus transacciones de commit

```
$ git config --global user.email "[email address]"
```

Habilita la útil colorización del producto de la línea de comando

```
$ git config --global color.ui auto
```

CREAR REPOSITORIOS

Inicia un nuevo repositorio u obtiene uno de una URL existente

Crea un nuevo repositorio local con el nombre especificado

```
$ git init [project-name]
```

Descarga un proyecto y toda su historia de versión

```
$ git clone [url]
```

Guía rápida de comandos

EFECTUAR CAMBIOS

Revisa las ediciones y elabora una transacción de commit

Enumera todos los archivos nuevos o modificados que se deben confirmar

```
$ git status
```

Muestra las diferencias de archivos que no se han enviado aún al área de espera

```
$ git diff
```

Toma una instantánea del archivo para preparar la versión

```
$ git add [file]
```

Muestra las diferencias del archivo entre el área de espera y la última versión del archivo

```
$ git diff --staged
```

Mueve el archivo del área de espera, pero preserva su contenido

```
$ git reset [file]
```

Registra las instantáneas del archivo permanentemente en el historial de versión

```
$ git commit -m "[descriptive message]"
```

Guía rápida de comandos

CAMBIOS GRUPALES

Nombra una serie de commits y combina esfuerzos ya culminados

Enumera todas las ramas en el repositorio actual

```
$ git branch
```

Crea una nueva rama

```
$ git branch [branch-name]
```

Cambia a la rama especificada y actualiza el directorio activo

```
$ git checkout [branch-name]
```

Borra la rama especificada

```
$ git branch [branch-name]
```

Combina el historial de la rama especificada con la rama actual

```
$ git branch -d [branch-name]
```

REHACER COMMITS

Borra errores y elabora historial de reemplazo

Deshace todos los commits después de [commit], preservando los cambios localmente

```
$ git reset [commit]
```

Desecha todo el historial y regresa al commit especificado

```
$ git reset --hard [commit]
```

Guía rápida de comandos

NOMBRES DEL ARCHIVO DE REFACTORIZACIÓN

Reubica y retira los archivos con versión

Borra el archivo del directorio activo y pone en el área de espera el archivo borrado

```
$ git rm [file]
```

Retira el archivo del control de versiones, pero preserva el archivo a nivel local

```
$ git rm --cached [file]
```

Cambia el nombre del archivo y lo prepara para commit

```
$ git mv [file-original] [file-renamed]
```

Muestra las diferencias del archivo entre el área de espera y la última versión del archivo

```
$ git diff --staged
```

SUPRIMIR TRACKING

Excluye los archivos temporales y las rutas

Un archivo de texto llamado `.gitignore` suprime la creación accidental de versiones de archivos y rutas que concuerdan con los patrones especificados

```
*.log build/ temp-*
```

Enumera todos los archivos ignorados en este proyecto

```
$ git ls-files --other --ignored --exclude-standard
```

Guía rápida de comandos

GUARDAR FRAGMENTOS

Almacena y restaura cambios incompletos

Almacena temporalmente todos los archivos tracked modificados

```
$ git stash
```

Restaura los archivos guardados más recientemente

```
$ git stash pop
```

Enumera todos los sets de cambios en guardado rápido

```
$ git stash list
```

Elimina el set de cambios en guardado rápido más reciente

```
$ git stash drop
```

REPASAR HISTORIAL

Navega e inspecciona la evolución de los archivos de proyecto

Enumera el historial de la versión para la rama actual

```
$ git log
```

Muestra las diferencias de contenido entre dos ramas

```
$ git diff [first-branch]...[second-branch]
```

Produce metadatos y cambios de contenido del commit especificado

```
$ git show [commit]
```

Guía rápida de comandos

SINCRONIZAR CAMBIOS

Registrar un marcador de repositorio e intercambiar historial de versión

Descarga todo el historial del marcador del repositorio

```
$ git fetch [bookmark]
```

Combina la rama del marcador con la rama local actual

```
$ git merge [bookmark]/[branch]
```

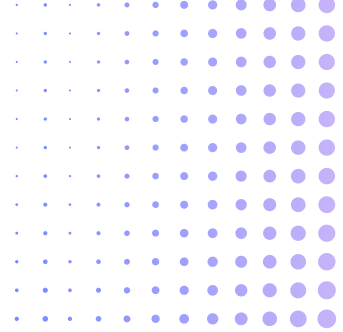
Carga todos los commits de la rama local al GitHub

```
$ git push [alias] [branch]
```

Descarga el historial del marcador e incorpora cambios

```
$ git pull
```

Beneficios de usar GITHUB



Accesibilidad y almacenamiento centralizado

Con GitHub, tus repositorios están en la nube, lo que significa que puedes acceder a ellos desde cualquier lugar con conexión a Internet. Esto lo hace ideal para equipos remotos o distribuidos, garantizando que todos trabajen con la versión más reciente del proyecto.

Portafolio y visibilidad

Para desarrolladores, GitHub actúa como una vitrina profesional. Los repositorios públicos permiten mostrar tus habilidades y proyectos a posibles empleadores o colaboradores, mejorando tu visibilidad en la industria tecnológica.

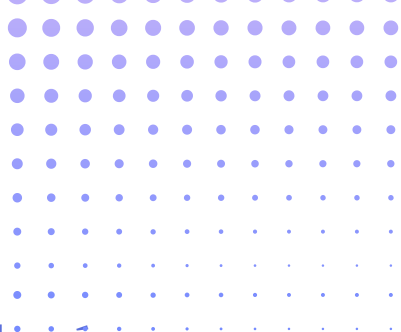
Documentación y soporte comunitario

Además de ser una herramienta de desarrollo, GitHub facilita la documentación de proyectos mediante wikis y archivos README. Sumado a esto, su gran comunidad de desarrolladores proporciona un espacio para aprender, colaborar y resolver dudas.

Gestión de proyectos eficiente

GitHub incluye herramientas como Issues, Project Boards y Discussions que permiten organizar tareas, planificar entregas y centralizar la comunicación del equipo, lo que simplifica la gestión de proyectos.





A lo largo de este handbook, hemos explorado cómo GitHub puede convertirse en el núcleo de tus flujos de trabajo, promoviendo la colaboración, la organización y la eficiencia en cualquier proyecto.

Desde la colaboración en equipo hasta el uso eficiente de herramientas y automatizaciones, GitHub es una plataforma poderosa que puede transformar la manera en que gestionas y desarrollas tus ideas.

Recomendaciones para el futuro

Colabora efectivamente: Utiliza Issues y Pull Requests para mantener una comunicación clara y mejorar el trabajo en equipo.

Automatiza procesos: Configura GitHub Actions para agilizar pruebas, despliegues y tareas repetitivas.

Sigue aprendiendo: Explora nuevas funcionalidades de GitHub y mantente actualizado con recursos oficiales y la comunidad.

Contribuye al código abierto: Participar en proyectos globales fortalecerá tus habilidades y ampliará tu red profesional.

Documenta bien: Mantén guías claras y actualizadas en tus repositorios para facilitar la colaboración y el futuro mantenimiento.

Quiero agradecerte por dedicar tiempo a leer este handbook y por tu interés en mejorar tus habilidades con GitHub.. Cada vez que decides aprender algo nuevo, fortaleces no solo tu futuro profesional, sino también la capacidad de tu equipo y comunidad para innovar.

GitHub no es solo una herramienta de desarrollo, sino un espacio donde ideas, código y personas convergen para resolver problemas, innovar y construir juntos.